NO.1 A provider configuration block is required in every Terraform configuration.
Example:

```
provider "provider_name" {
   ...
}
```

A. True
B. False
Answer: B
Explanation:
A provider configuration block is not required in every Terraform configuration. A provider configuration block can be omitted if its contents would otherwise be empty. Terraform assumes an empty default configuration for any provider that is not explicitly configured. However, some providers may require some configuration arguments (such as endpoint URLs or cloud regions) before they can be used. A provider's documentation should list which configuration arguments it expects. For providers distributed on the Terraform Registry, versioned documentation is available on each provider's page, via the "Documentation" link in the provider's header1. References = [Provider Configuration]1

NO.2 Terraform can only manage resource dependencies if you set them explicitly with the depends_on argument.
A. True
B. False
Answer: B
Explanation:
Terraform can manage resource dependencies implicitly or explicitly. Implicit dependencies are created when a resource references another resource or data source in its arguments. Terraform can infer the dependency from the reference and create or destroy the resources in the correct order. Explicit dependencies are created when you use the depends_on argument to specify that a resource depends on another resource or module.
This is useful when Terraform cannot infer the dependency from the configuration or when you need to create a dependency for some reason outside of Terraform's scope. References = : Create resource dependencies : Terraform Resource Dependencies Explained

NO.3 If you manually destroy infrastructure, what is the best practice reflecting this change in Terraform?
A. Run terraform refresh
B. It will happen automatically
C. Manually update the state fire
D. Run terraform import
Answer: B

Explanation:
If you manually destroy infrastructure, Terraform will automatically detect the change and update the state file during the next plan or apply. Terraform compares the current state of the infrastructure with the desired state in the configuration and generates a plan to reconcile the differences. If a resource is missing from the infrastructure but still exists in the state file, Terraform will attempt to recreate it. If a resource is present in the infrastructure but not in the state file, Terraform will ignore it unless you use the terraform import command to bring it under Terraform's management. References = [Terraform State]

NO.4 Which command add existing resources into Terraform state?
A. Terraform init
B. Terraform plan
C. Terraform refresh
D. Terraform import
E. All of these
Answer: D
Explanation:
This is the command that can add existing resources into Terraform state, by matching them with the corresponding configuration blocks in your files.

NO.5 Module version is required to reference a module on the Terraform Module Registry.
A. True
B. False
Answer: B
Explanation:
Module version is optional to reference a module on the Terraform Module Registry. If you omit the version constraint, Terraform will automatically use the latest available version of the module

NO.6 What does the default "local" Terraform backend store?
A. tfplan files
B. State file
C. Provider plugins
D. Terraform binary
Answer: B
Explanation:
The default "local" Terraform backend stores the state file in a local file named terraform.tfstate, which can be used to track and manage the state of your infrastructure3.

NO.7 Which are forbidden actions when the terraform state file is locked? Choose three correct answers.
A. Terraform state list
B. Terraform destroy
C. Terraform validate
D. Terraform validate

E. Terraform for
F. Terraform apply
Answer: B C F
Explanation:
The terraform state file is locked when a Terraform operation that could write state is in progress. This prevents concurrent state operations that could corrupt the state. The forbidden actions when the state file is locked are those that could write state, such as terraform apply, terraform destroy, terraform refresh, terraform taint, terraform untaint, terraform import, and terraform state *. The terraform validate command is also forbidden, because it requires an initialized working directory with the state file. The allowed actions when the state file is locked are those that only read state, such as terraform plan, terraform show, terraform output, and terraform console. References = [State Locking] and [Command: validate]

NO.8 You have created a main.tf Terraform configuration consisting of an application server, a database and a load balanced. You ran terraform apply and Terraform created all of the resources successfully.
Now you realize that you do not actually need the load balancer, so you run terraform destroy without any flags. What will happen?
A. Terraform will prompt you to pick which resource you want to destroy
B. Terraform will destroy the application server because it is listed first in the code
C. Terraform will prompt you to confirm that you want to destroy all the infrastructure
D. Terraform will destroy the main, tf file
E. Terraform will immediately destroy all the infrastructure
Answer: C
Explanation:
This is what will happen if you run terraform destroy without any flags, as it will attempt to delete all the resources that are associated with your current working directory or workspace. You can use the -target flag to specify a particular resource that you want to destroy.

NO.9 Which method for sharing Terraform configurations fulfills the following criteria:
1. Keeps the configurations confidential within your organization
2. Support Terraform's semantic version constrains
3. Provides a browsable directory
A. Subfolder within a workspace
B. Generic git repository
C. Terraform Cloud private registry
D. Public Terraform module registry
Answer: C
Explanation:
This is the method for sharing Terraform configurations that fulfills the following criteria:
Keeps the configurations confidential within your organization
Supports Terraform's semantic version constraints
Provides a browsable directory
The Terraform Cloud private registry is a feature of Terraform Cloud that allows you to host and manage your own modules within your organization, and use them in your Terraform configurations

with versioning and access control.

NO.10 While attempting to deploy resources into your cloud provider using Terraform, you begin to see some odd behavior and experience slow responses. In order to troubleshoot you decide to turn on Terraform debugging.
Which environment variables must be configured to make Terraform's logging more verbose?
A. TF_LOG_PAIH
B. TF_LOG
C. TF_VAR_log_path
D. TF_VAR_log_level
Answer: B
Explanation:
To make Terraform's logging more verbose for troubleshooting purposes, you must configure the TF_LOG environment variable. This variable controls the level of logging and can be set to TRACE, DEBUG, INFO, WARN, or ERROR, with TRACE providing the most verbose output.References = Detailed debugging instructions and the use of environment variables like TF_LOG for increasing verbosity are part of Terraform's standard debugging practices

NO.11 How can a ticket-based system slow down infrastructure provisioning and limit the ability to scale? Choose two correct answers.
A. End-users have to request infrastructure changes
B. Ticket based systems generate a full audit trail of the request and fulfillment process
C. Users can access catalog of approved resources from drop down list in a request form
D. The more resources your organization needs, the more tickets your infrastructure team has to process
Answer: A
Explanation:
These are some of the ways that a ticket-based system can slow down infrastructure provisioning and limit the ability to scale, as they introduce delays, bottlenecks, and manual interventions in the process of creating and modifying infrastructure.

NO.12 You decide to move a Terraform state file to Amazon S3 from another location. You write the code below into a file called backend.tf.

```
terraform {
  backend "s3" {
    bucket - "my-tf-bucket"
    region = "us-east-1"
  }
}
```

Which command will migrate your current state file to the new S3 remote backend?
A. terraform state

B. terraform init
C. terraform push
D. terraform refresh
Answer: B
Explanation:
This command will initialize the new backend and prompt you to migrate the existing state file to the new location3. The other commands are not relevant for this task.

NO.13 A developer on your team is going lo leaf down an existing deployment managed by Terraform and deploy a new one. However, there is a server resource named aws instant.ubuntu[l] they would like to keep. What command should they use to tell Terraform to stop managing that specific resource?
A. Terraform plan rm:aws_instance.ubuntu[1]
B. Terraform state rm:aws_instance.ubuntu[1]
C. Terraform apply rm:aws_instance.ubuntu[1]
D. Terraform destory rm:aws_instance.ubuntu[1]
Answer: B
Explanation:
To tell Terraform to stop managing a specific resource without destroying it, you can use the terraform state rm command. This command will remove the resource from the Terraform state, which means that Terraform will no longer track or update the corresponding remote object. However, the object will still exist in the remote system and you can later use terraform import to start managing it again in a different configuration or workspace. The syntax for this command is terraform state rm <address>, where <address> is the resource address that identifies the resource instance to remove. For example, terraform state rm aws_instance.ubuntu
[1] will remove the second instance of the aws_instance resource named ubuntu from the state.
References: Command: state rm : Moving Resources

NO.14 Which type of block fetches or computes information for use elsewhere in a Terraform configuration?
A. data
B. local
C. resource
D. provider
Answer: A
Explanation:
Detailed Explanation:
* Rationale for Correct Answer (A):data blocks fetch information from providers (like AMI IDs, network info) that can be used in other resources.
* Analysis of Incorrect Options:
* B. local: Stores values, doesn't fetch external data.
* C. resource: Defines managed infrastructure, not read-only data.
* D. provider: Configures connection to APIs, not data fetching.
* Key Concept:Data sources are read-only queries into existing infrastructure.
Reference:Terraform Exam Objective - Read, Generate, and Modify Configurations.

NO.15 You have never used Terraform before and would like to test it out using a shared team account for a cloud provider. The shared team account already contains 15 virtual machines (VM). You develop a Terraform configuration containing one VM. perform terraform apply, and see that your VM was created successfully.
What should you do to delete the newly-created VM with Terraform?
A. The Terraform state file contains all 16 VMs in the team account. Execute terraform destroy and select the newly-created VM.
B. Delete the Terraform state file and execute terraform apply.
C. The Terraform state file only contains the one new VM. Execute terraform destroy.
D. Delete the VM using the cloud provider console and terraform apply to apply the changes to the Terraform state file.
Answer: C
Explanation:
This is the best way to delete the newly-created VM with Terraform, as it will only affect the resource that was created by your configuration and state file. The other options are either incorrect or inefficient.

NO.16 Which statement describes a goal of Infrastructure as Code (IaC)?
A. A pipeline process to test and deliver software.
B. Write once, run anywhere.
C. The programmatic configuration of resources.
D. Defining a vendor-agnostic API.
Answer: C
Explanation:
Infrastructure as Code (IaC) refers to managing infrastructureprogrammatically, enabling automation and repeatability.
Ais incorrect becauseIaC is not just about software delivery pipelines; it specifically deals with infrastructure.
Bis incorrect because"Write once, run anywhere" applies more to software development than IaC.
Dis incorrect becauseIaC does not enforce vendor-agnostic APIs; it depends on the tool and provider.
Official Terraform Documentation Reference:
What is Infrastructure as Code?

NO.17 Which configuration consistency errors does terraform validate report?
A. Terraform module isn't the latest version
B. Differences between local and remote state
C. Declaring a resource identifier more than once
D. A mix of spaces and tabs in configuration files
Answer: C
Explanation:
Terraform validate reports configuration consistency errors, such as declaring a resource identifier more than once. This means that the same resource type and name combination is used for multiple resource blocks, which is not allowed in Terraform. For example, resource "aws_instance" "example" {...} cannot be used more than once in the same configuration. Terraform validate does not report

errors related to module versions, state differences, or formatting issues, as these are not relevant for checking the configuration syntax and structure. References = [Validate Configuration], [Resource Syntax]

NO.18 What task does the terraform import command perform?
A. Imports resources from one Terraform state file to another.
B. Imports existing resources into Terraform's state file.
C. Imports a new Terraform module into Terraform's state file.
D. Imports all infrastructure from the configured cloud provider.
E. Imports provider configuration from one state file to another.
Answer: B
Explanation:
Detailed Explanation:
* Rationale for Correct Answer (B):terraform import allows Terraform to associate existing resources (created outside of Terraform or by another tool) with a Terraform configuration by writing them into the state file. After import, Terraform can manage those resources.
* Analysis of Incorrect Options:
* A. From one state file to another: Incorrect, import does not transfer between state files.
* C. Importing a module: Incorrect, modules are defined in configuration, not imported.
* D. Import all infrastructure: Incorrect, import is per-resource, not bulk.
* E. Provider configuration transfer: Incorrect, provider configs are in .tf files, not imported with this command.
* Key Concept:The terraform import command bridges existing resources with Terraform state management.
Reference:Terraform Exam Objective - Implement and Maintain State.

NO.19 You're building a CI/CD (continuous integration/continuous delivery) pipeline and need to inject sensitive variables into your Terraform run. How can you do this safely?
A. Copy the sensitive variables into your Terraform code
B. Store the sensitive variables in a secure_varS.tf file
C. Store the sensitive variables as plain text in a source code repository
D. Pass variables to Terraform with a -var flag
Answer: D
Explanation:
This is a secure way to inject sensitive variables into your Terraform run, as they will not be stored in any file or source code repository. You can also use environment variables or variable files with encryption to pass sensitive variables to Terraform.

NO.20 Which of the following is not a valid Terraform collection type?
A. Tree
B. Map
C. List
D. set
Answer: A

Explanation:
This is not a valid Terraform collection type, as Terraform only supports three collection types: list, map, and set. A tree is a data structure that consists of nodes with parent-child relationships, which is not supported by Terraform.

NO.21 Which Terraform command checks that your configuration syntax is correct?
A. terraform validate
B. terraform init
C. terraform show
D. terraform fmt
Answer: A
Explanation:
The terraform validate command is used to check that your Terraform configuration files are syntactically valid and internally consistent. It is a useful command for ensuring your Terraform code is error-free before applying any changes to your infrastructure.

NO.22 You modified your Terraform configuration to fix a typo in the resource ID by renaming it from photoes to photos. What configuration will you add to update the resource ID in state without destroying the existing resource?
Original configuration:
resource "aws_s3_bucket" "photoes" {
bucket_prefix = "images"
}
Updated configuration:
resource "aws_s3_bucket" "photos" {
bucket_prefix = "images"
}
A. moved {
from = aws_s3_bucket.photoes
to = aws_s3_bucket.photos
}
B. moved {
bucket.photoes = aws_s3_bucket.photos
}
C. moved {
aws_s3_bucket.photoes = aws_s3_bucket.photos
}
D. None. Terraform will automatically update the resource ID.
Answer: A
Explanation:
Detailed Explanation:
* Rationale for Correct Answer (A):Terraform does not automatically update state references when resource identifiers are renamed. Instead, you must use a moved block in your configuration to inform Terraform how to map the old resource to the new one. This prevents Terraform from destroying and recreating the resource.

* Analysis of Incorrect Options:
* B & C: Incorrect syntax - moved requires from and to.
* D: Incorrect, Terraform won't auto-detect renames and will plan to destroy and recreate the resource unless a moved block is provided.
* Key Concept:The moved block is essential for refactoring resource names without losing resources in state.
Reference:Terraform Exam Objective - Implement and Maintain State.

NO.23 A developer launched a VM outside of the Terraform workflow and ended up with two servers with the same name. They are unsure which VM is managed with Terraform, but they do have a list of all active VM IDs.
Which method could you use to determine which instance Terraform manages?
A. Modify the Terraform configuration to add an import block for both of the virtual machines.
B. Run a terraform apply -refresh to identify the virtual machine IDs that are already managed by Terraform.
C. Run terraform state rm on both VMs, then terraform apply to recreate the correct one.
D. Run terraform state list to find the names of all VMs, then run terraform state show for each of them to find which VM ID Terraform manages.
Answer: D
Explanation:
Detailed Explanation:
* Rationale for Correct Answer (D):terraform state list shows all resources currently managed by Terraform. terraform state show <resource> provides detailed attributes, including the VM ID. This lets you match the Terraform-managed instance with the actual infrastructure.
* Analysis of Incorrect Options:
* A: Importing is not needed since one VM is already in state.
* B: terraform apply doesn't show which VM ID is managed, it only refreshes attributes.
* C: Removing state entries is destructive and may lead to losing state data unnecessarily.
* Key Concept:The state file is Terraform's source of truth for which resources it manages.
Reference:Terraform Exam Objective - Implement and Maintain State.

NO.24 You modified your Terraform configuration and run Terraform plan to review the changes. Simultaneously, your teammate manually modified the infrastructure component you are working on. Since you already ran terraform plan locally, the execution plan for terraform apply will be the same.
A. True
B. False
Answer: B
Explanation:
The execution plan for terraform apply will not be the same as the one you ran locally with terraform plan, if your teammate manually modified the infrastructure component you are working on. This is because Terraform will refresh the state file before applying any changes, and will detect any differences between the state and the real resources.

NO.25 Any user can publish modules to the public Terraform Module Registry.

A. True

B. False

Answer: A

Explanation:

The Terraform Registry allows any user to publish and share modules. Published modules support versioning, automatically generate documentation, allow browsing version histories, show examples and READMEs, and more. Public modules are managed via Git and GitHub, and publishing a module takes only a few minutes. Once a module is published, releasing a new version of a module is as simple as pushing a properly formed Git tag1.

References = The information can be verified from the Terraform Registry documentation on Publishing Modules provided by HashiCorp Developer1.

NO.26 If you update the version constraint in your Terraform configuration, Terraform will update your lock file the next time you run terraform Init.

A. True

B. False

Answer: A

Explanation:

If you update the version constraint in your Terraform configuration, Terraform will update your lock file the next time you run terraform init3. This will ensure that you use the same provider versions across different machines and runs.